

OpenCloudOS 内存管理与卸载技术的探究与实践

宋恺睿

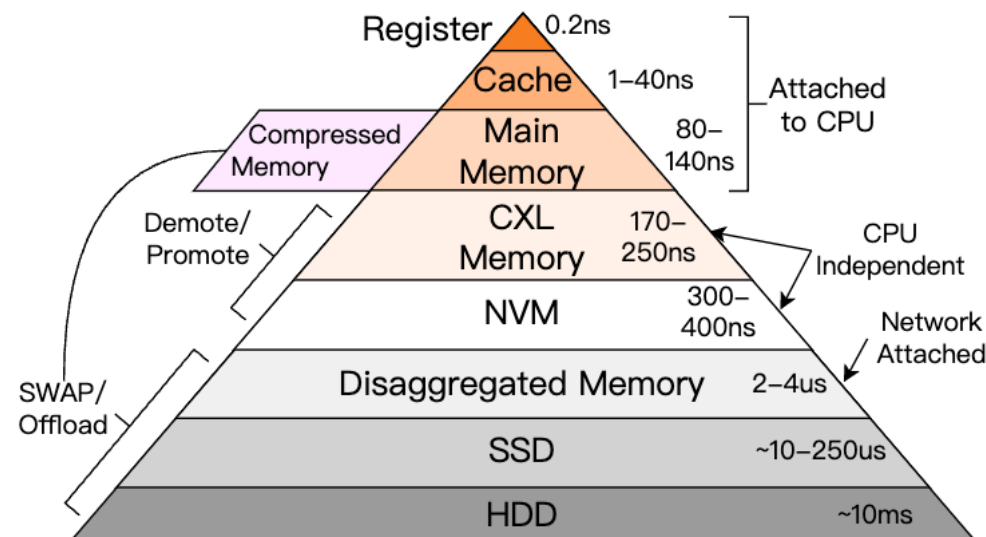
2025-03-29

内容提要

- 内存管理与内存多级卸载
- 为什么要让操作系统做主动内存卸载
- 业界痛点与我们的突破
- 未来展望

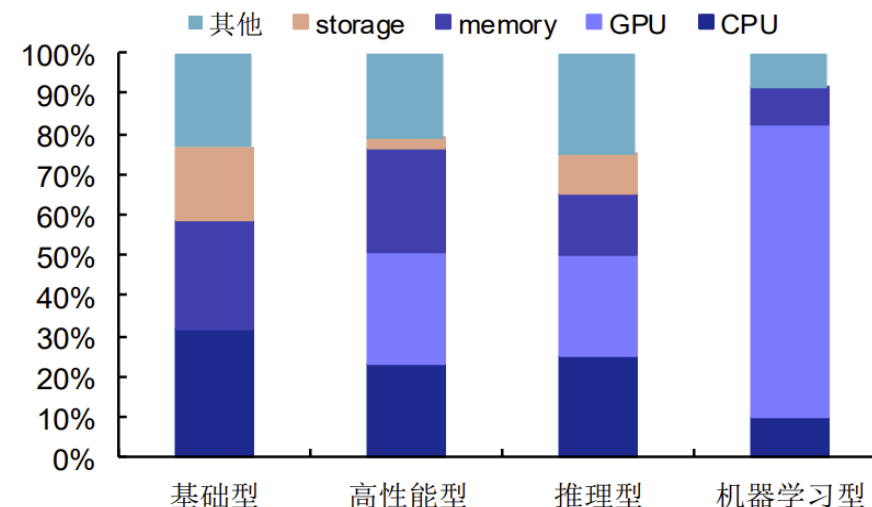
内存管理与内存多级卸载

- 业务与数据规模复杂度不断增长，**内存成本，管理复杂性**日渐增长。
- 在数据中心，内存是数据中心的主要核心成本之一。在桌面与移动平台，内存更是紧缺资源。
- 系统中往往是存在长期不访问的冷内存，或者相对较冷内存，**内存卸载**便是要充分利用整机所有资源，将冷内存移动到次级存储设备。
- 如何进行更好的内存管理卸载以提高性能，降低成本，一直是业界探索的热门方向。
 - 存在多级存储的设备上对内存进行合理沉降。
 - 借助高性能算法实现内存压缩。
 - 同时可以提前预防内存巅峰压力的形成。



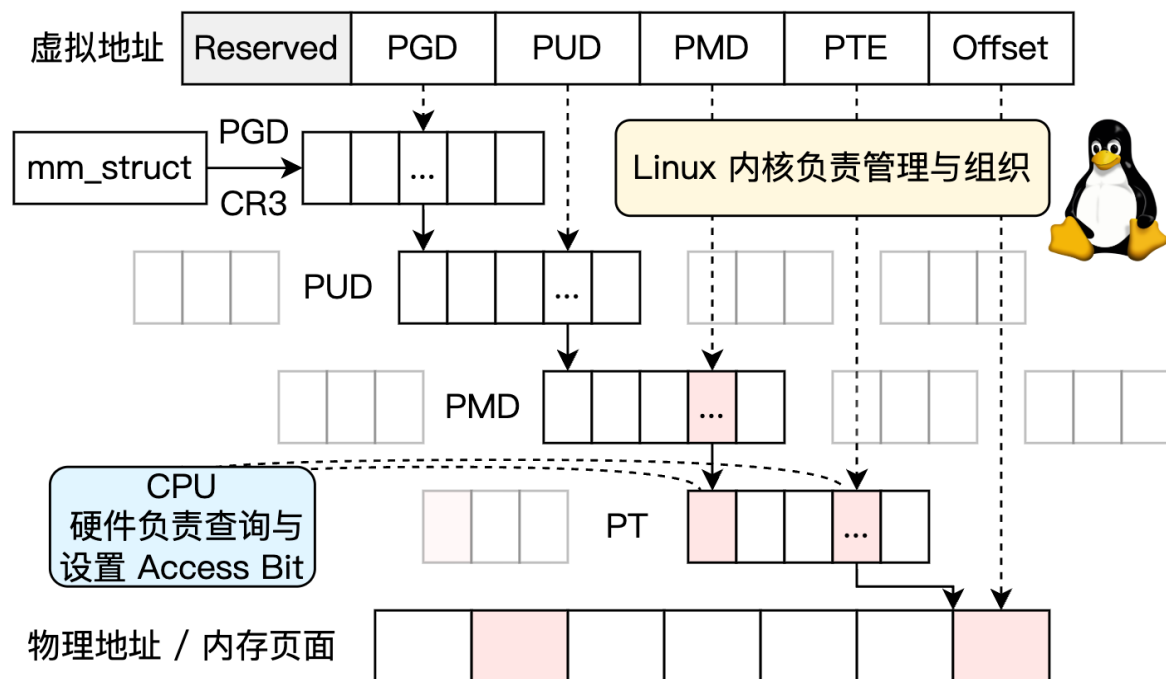
图片基于 Meta TMO 论文修改

图 35：各类服务器成本构成



资料来源：IDC，安信证券研究中心

为什么要让操作系统做主动内存卸载



- 操作系统的最底层：内核
- 内核最主要的核心功能 - 虚拟内存
 - 管理页表
 - 缺页处理
 - Over commit
 - 迁移 / 整理 / 页面分配 / ...
- 自然而然内核可以充分利用 CPU 等硬件提供的内存控制，感知能力，映射等能力，统一调度全局所有资源。
 - 热度探测
 - Demote
 - SWAP

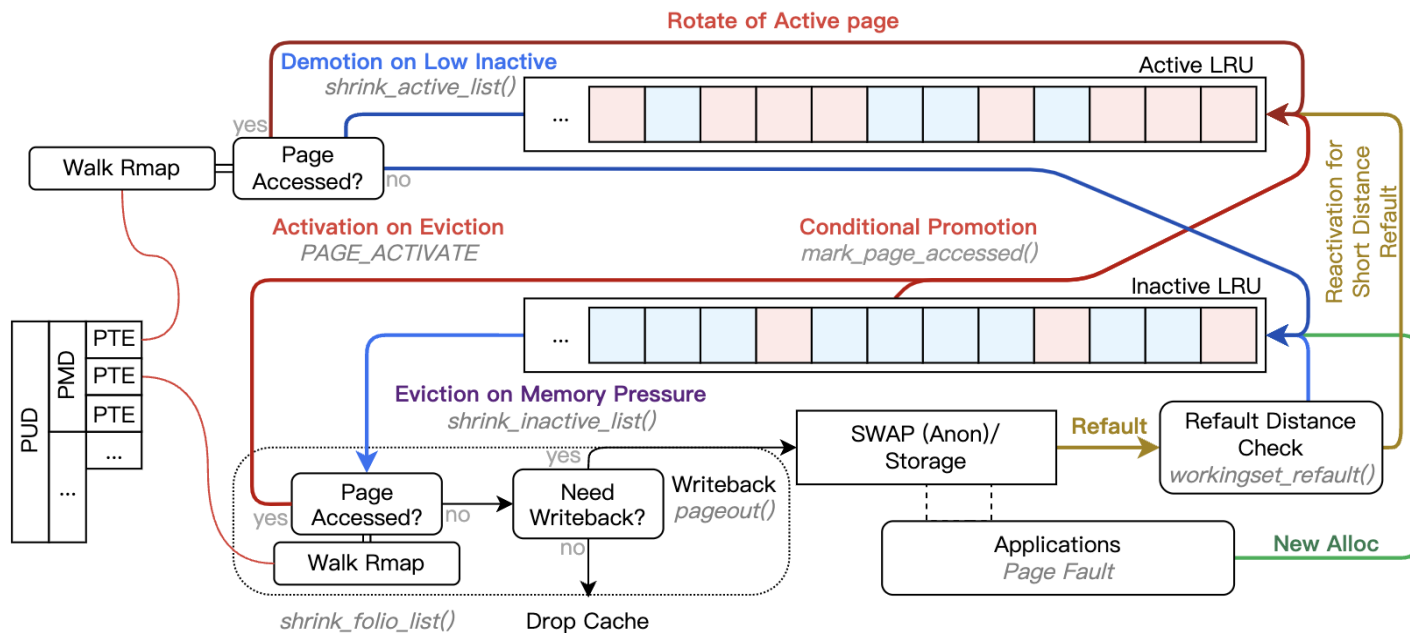
业界痛点, 现状, 与突破

业界痛点，现状，与突破

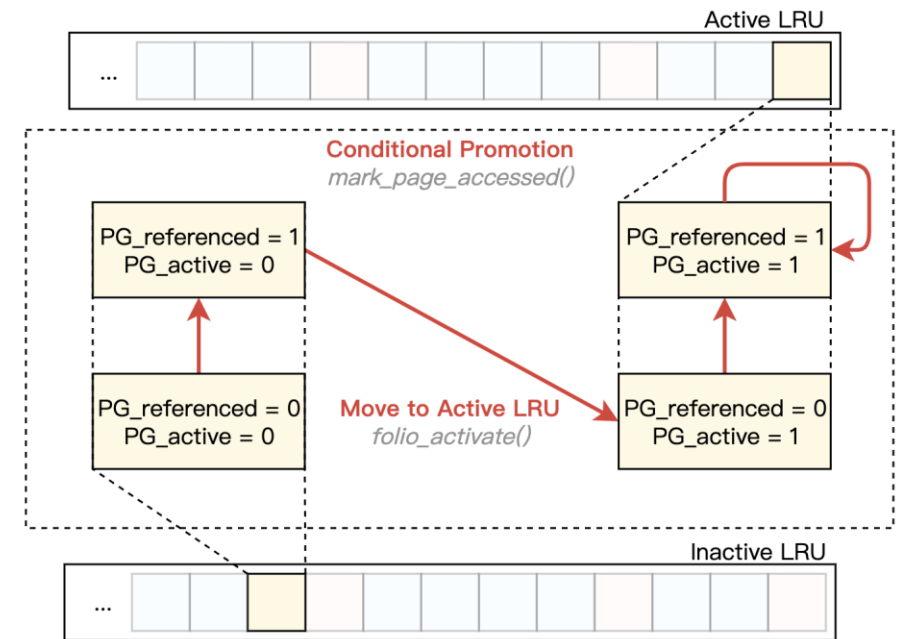
- **热度探测**：如何在生产环境中找出冷内存？如何平衡精度与性能？
 - **内存卸载**：匿名页 -> SWAP 系统的瓶颈在哪里？缓存 -> 回收的效率？
 - **mTHP 管理**：社区引入了新的 mTHP 概念（支持不同大小的大页），带来了更多挑战？
 - **内存管理开销优化**：内核的内存的管理提供了大量特性，这必然是有开销的，如何降到最低？特别是随着系统规模增长如何保证可扩展性（Scalability）？
-
- 针对内存管理与卸载，我们进行了大量探索与实践，并实现了一系列创新成果。
 - 所有技术进步方案全部回馈社区，在 Linux 内核主线中我们已经在内存管理领域做出大量贡献。
 - 推动 OpenCloudOS 与整个 Linux 社区的技术进步。

热度探测 - 冷内存识别与淘汰

- Linux 内核管理内存页面的核心机制：LRU，不仅管理热度，也是组织已分配页面的基本结构。
- 其中“U”的主要判断来源之一便是页表 Access Bit，如何平衡扫描页表收集热度信息的开销与精度？
- Linux 内核经典 **Active/Inactive LRU**；发生压力时逐页降级/淘汰与扫描；**RMAP 开销大，精度有限**。
- 近年来社区提供了各种新方式方法进行热度探测：如 DAMON，Idle Page 等。
- 但 LRU 仍旧是最重要最通用核心组件，基于 LRU 的通用优化是全场景内存卸载的核心。



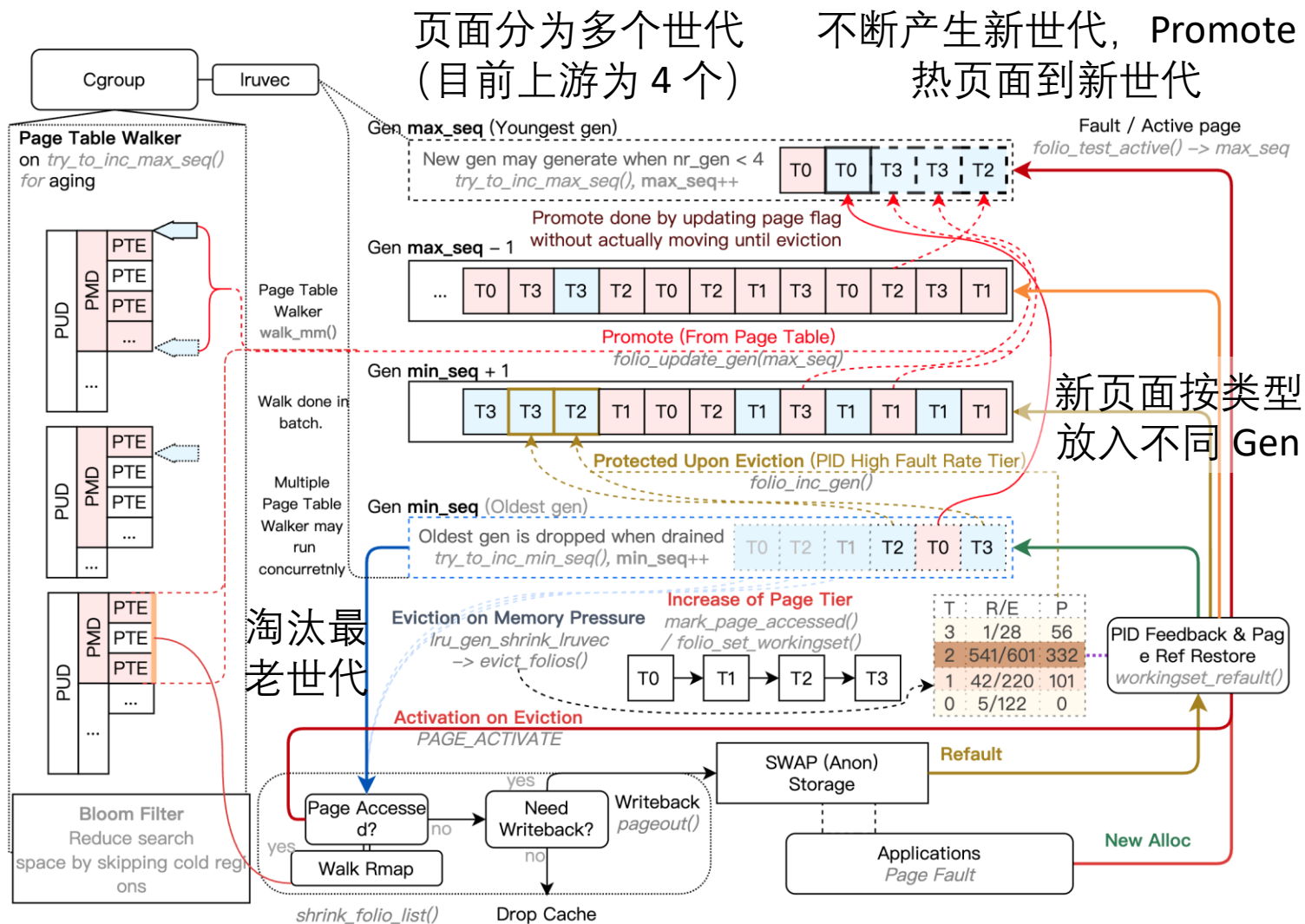
页面分为 Active / Inactive 两级，有内存压力时，移动页面从 Active -> Inactive 或淘汰。每次移动 / 淘汰走 RMAP 扫描页表。



非 Map 页面通过 `folio_mark_accessed()` 进行 LRU 升级 (Inactive -> Active)。

热度探测 - 冷内存识别与淘汰

- 谷歌在 6.1 中提出了 MGLRU 机制，推动了内核内存管理方向性能和特性的大进步：
- 核心理念为：尽量避免了 RMAP，而尽量使用 Page Table Walk 和页面热度升级作为主要机制。
- 理念上，MGLRU 会不断产生新世代，并将当前探测到的热页放入新世代中。目前上游最多 4 个世代。
- 内存出现压力时，直接消耗最冷世代中的页面，没有页面在 LRU 中页面降级，同时启动 Table Walker。
- 页面引入 Tier 概念来统计来自内核 `folio_mark_accessed()` 等热度信息。
- 引入 PID 等控制算法平衡不同类型的页面回收与保护特定 Tier 页面。



热度探测 - 冷内存识别与淘汰

- 我们针对 MGLRU 展开了一系列探索和实践。
- 6.1 版本中 MGLRU 有着比较明显的文件页回收倾向，这让文件页较热场景中，性能不够理想。
- 我们重新设计了源自 Active / Inactive LRU 的 Workingset Refault Distance 算法，使其适配 MGLRU，在上游修复 OOM Regression 相关问题与推进算法改进。

After this patch, some typical client and server workloads showed improvements under heavy memory pressure. For example, Python TPC-C, which was used to benchmark a different approach [1] to better detect refault distances, showed a significant decrease in total refaults:

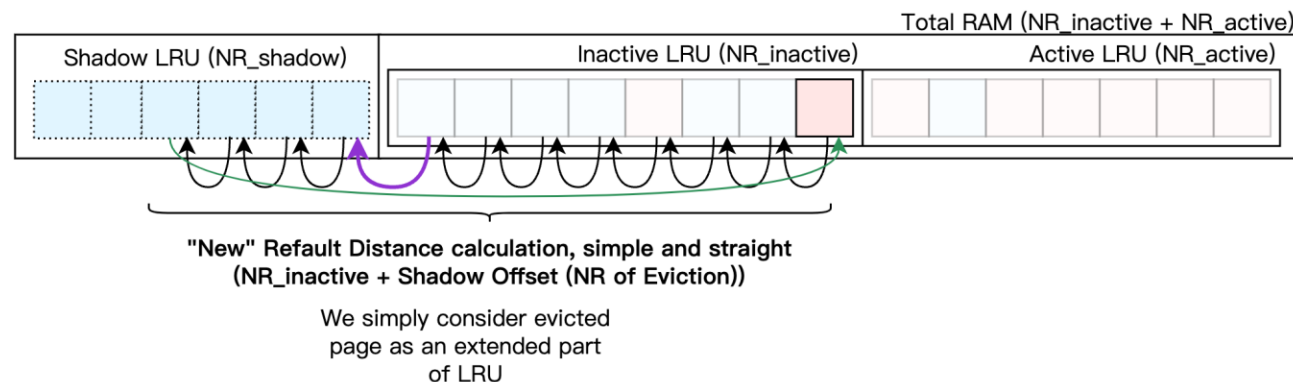
	Before	After	Change
Time (seconds)	10801	10801	0%
Executed (transactions)	41472	43663	+5%
workingset_nodes	109070	120244	+10%
workingset_refault_anon	5019627	7281831	+45%
workingset_refault_file	1294678786	554855564	-57%
workingset_refault_total	1299698413	562137395	-57%

[1] <https://lore.kernel.org/20230920190244.16839-1-ryncsn@gmail.com/>

Reported-by: Kairui Song <kasong@tencent.com>

上游最新算法改进：

<https://lore.kernel.org/lkml/20241206003126.1338283-7-yuzhao@google.com>



继承自 Active/Inactive LRU 的算法：每个换出页面分配一个时间戳（8 bytes），每个 LRU 维护一个换出计数时间戳（atomic），比较两者即可得知换出页面的热度。

```
Before (with ZRAM enabled, the result won't change whether
any kind of swap is on or not):
$ tpcc.py --config=mongodb.config mongodb --duration=900 --warehouses=500 --clients=30
=====
Execution Results after 919 seconds
=====
STOCK_LEVEL    Executed    Time (µs)    Rate
-----
TOTAL          577        27584645283.7  0.02 txn/s

Patched (with ZRAM enabled):
$ tpcc.py --config=mongodb.config mongodb --duration=900 --warehouses=500 --clients=30
=====
Execution Results after 905 seconds
=====
STOCK_LEVEL    Executed    Time (µs)    Rate
-----
TOTAL          2542       27121571486.2  0.09 txn/s
```

算法提交上游，部分工作负载有极大提升：

<https://lore.kernel.org/all/20230920190244.16839-3-ryncsn@gmail.com>

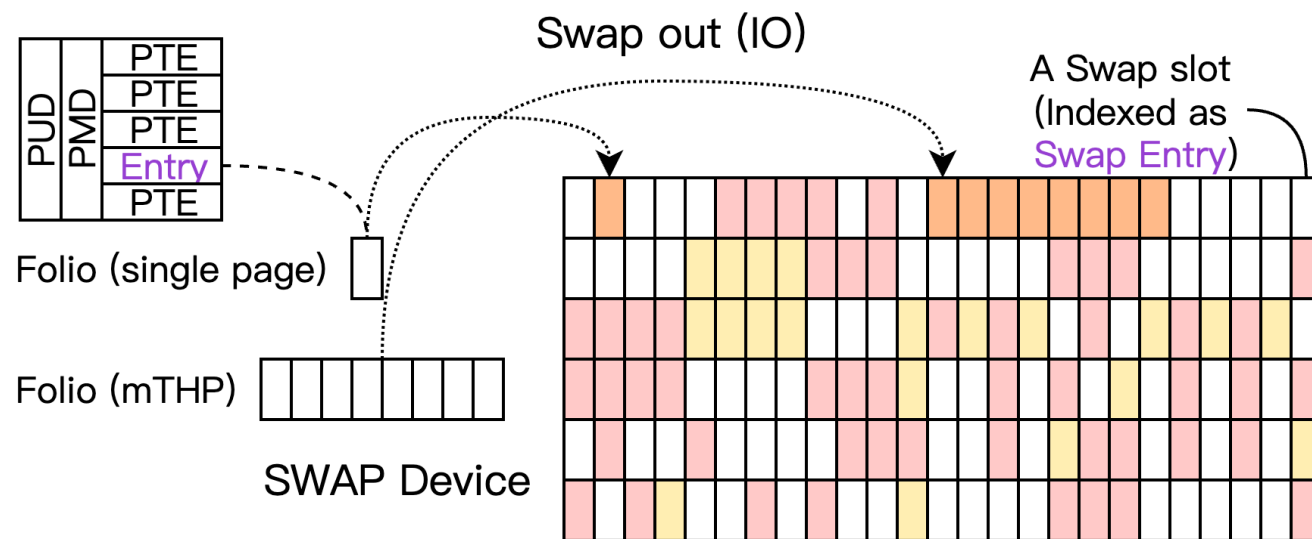
SWAP 分配

- 对于缓存文件页面，淘汰时可以直接丢弃。而应用程序内存（匿名页面）是内存卸载的主要挑战。
- 在没有 Demote 的场景下，匿名页面的卸载便主要依赖 SWAP，将内存页放入块设备。
- SWAP 分配是 SWAP 环节中的核心步骤之一，到 6.11 为止，其也是主要性能阻碍之一。
- Swap 设备存储单一页面（4K）的单位为 **Swap Slot**，内核使用一个大的 Map（**Swap Map** 1 byte per slot）——对应记录与管理 Swap Slot 使用情况。内存页表中管理 Swap Map / Slot 的索引基础数据为 **Swap Entry**（**Swap Entry** -> **Swap Map / Slot** 一一对应）。
- 在可变大页场景中，Swap Allocator 会因为无法找到连续的 Slot 导致大页换出失败。

With Barry's mthp test program V2:

```
Without:
$ ./thp_swap_allocator_test -a
Iteration 1: swpout inc: 32, swpout fallback inc: 192, Fallback percentage: 85.71%
Iteration 2: swpout inc: 0, swpout fallback inc: 231, Fallback percentage: 100.00%
Iteration 3: swpout inc: 0, swpout fallback inc: 227, Fallback percentage: 100.00%
...
Iteration 98: swpout inc: 0, swpout fallback inc: 224, Fallback percentage: 100.00%
Iteration 99: swpout inc: 0, swpout fallback inc: 215, Fallback percentage: 100.00%
Iteration 100: swpout inc: 0, swpout fallback inc: 222, Fallback percentage: 100.00%
```

上游 mTHP 测试工具显示 6.11 内核中，随时间推移 SWAP 大页分配失败率接近 100%



SWAP 效率

- SWAP 分配器长期以来保持极低的内存使用（1 byte per entry / swaped out page）。
- 针对 SSD 和 HDD 各有优化，同时使用了如 Slot Cache 等机制来进一步提高性能。
- 但随着系统规模扩大以及包括 ZRAM 在内的高性能 SWAP 设备的出现，现有机制与算法开始显出不足。
- 以 Build Linux Kernel 作为典型场景，内存压力下，SWAP 分配器会导致分钟级的累积全局阻塞，开销占据总开销一半以上。

```
`perf lock` indicates this issue clearly  
using tmpfs and ZRAM with limited memory  
(pages), result of "perf lock contention"
```

contended	total wait	max wait
34948	53.63 s	7.11 ms
16569	40.05 s	6.45 ms
11191	28.41 s	7.03 ms
4147	22.78 s	122.66 ms
4595	7.17 s	6.79 ms
406027	2.74 s	2.59 ms
...snip...		

```
The time consumption and stack trace of a page fault before:  
- 78.45% 0.17% cc1 [kernel.kallsyms]  
[k] asm_exc_page_fault  
- 78.28% asm_exc_page_fault  
- 78.18% exc_page_fault  
- 78.17% do_user_addr_fault  
- 78.09% handle_mm_fault  
- 78.06% __handle_mm_fault  
- 69.69% do_swap_page  
- 55.87% alloc_swap_folio  
- 5.49 ms spinlock page_vma_mapped_walk+0x6f3  
- 1.56 ms spinlock swapcache_free_entries+0x59  
- 6.74 us spinlock list_lru_add+0x39
```

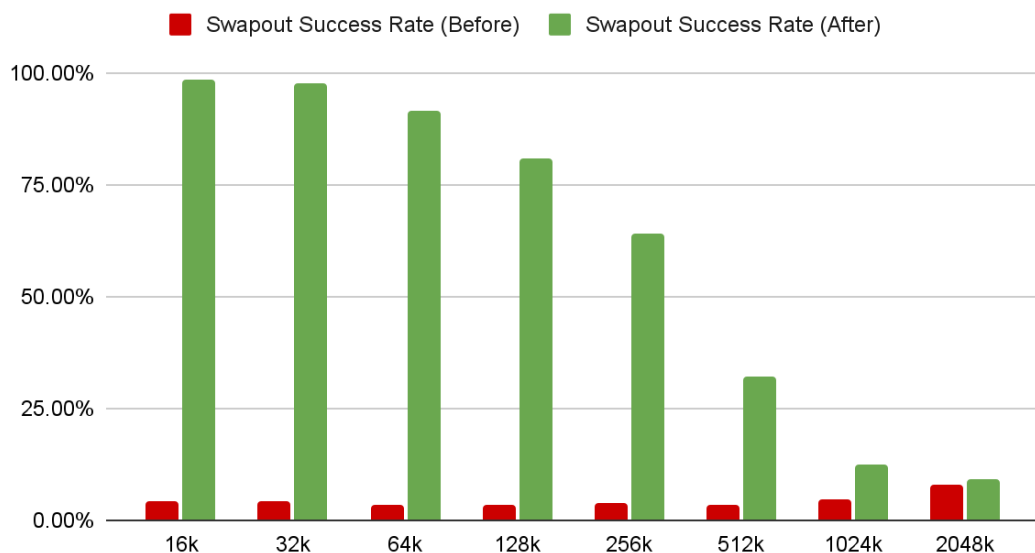
- 6.12 中已合入：

A poster for the Linux Plumbers Conference 2024. The top half features a dark blue background with silhouettes of trees and a red light. The text 'LINUX PLUMBERS CONFERENCE' is in large, white, sans-serif capital letters. Below it, 'Vienna, Austria' and 'September 18-20, 2024' are in smaller white text. A horizontal line separates this from a dark grey section. In the grey section, 'zone' is written in white on the left. The title 'mTHP and swap allocator' is in a large, yellow-green, sans-serif font. Below the title, a calendar icon is followed by 'Sep 20, 2024, 5:15 PM'. A clock icon is followed by '15m'. A location pin icon is followed by '"Hall L1" (Austria Center)'. The word 'Speakers' is in a purple, sans-serif font. At the bottom, two speaker entries are shown: 'Chris Li (Google)' and 'Kairui Song (Tencent)', each preceded by a blue person icon.

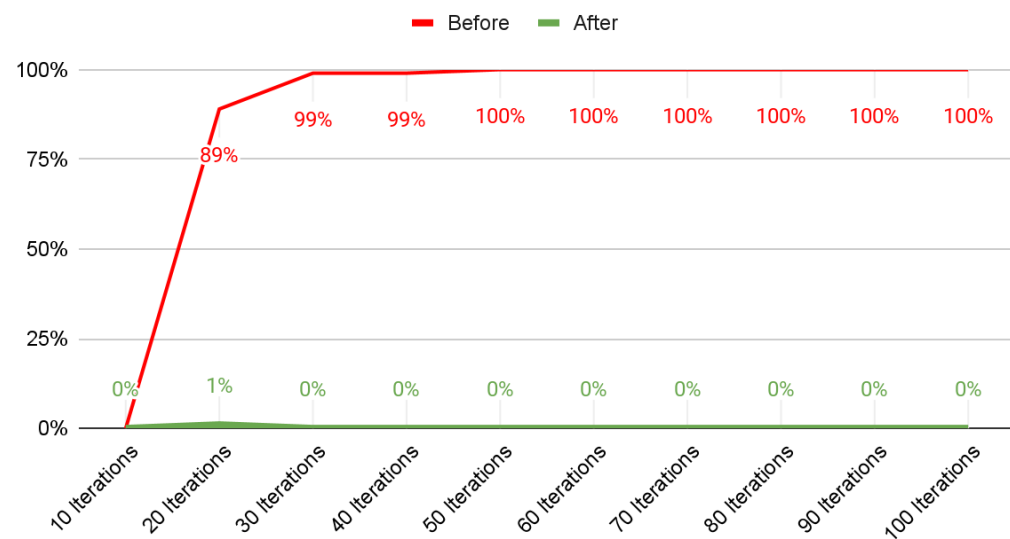
SWAP

- 在 6.12 内核中，Google 与我们一同作为共同作者，展开了上游合作，重构 SWAP 分配器。
- 老分配器以 Cluster（2M）作为 Hint 分配 THP 页面，新分配器以 **Cluster 作为基础管理单位**。
- 完全基于 Cluster 的管理过程。使用双链表将**全空，非空，碎片化，全满** Cluster 分级管理。
- 统一分配流程，避免直接 SWAP Map 扫描，以 Cluster 为单位进行索引，只扫描空闲 Cluster。
- 每个 Order 均有独立 Cluster 链表，避免不同 Order 的分配相互干扰，对 mTHP 友好。
- 大部分场景下消除了高 Order 分配时的碎片化问题。

Build Linux Kernel with mTHP enabled (single type of mTHP)

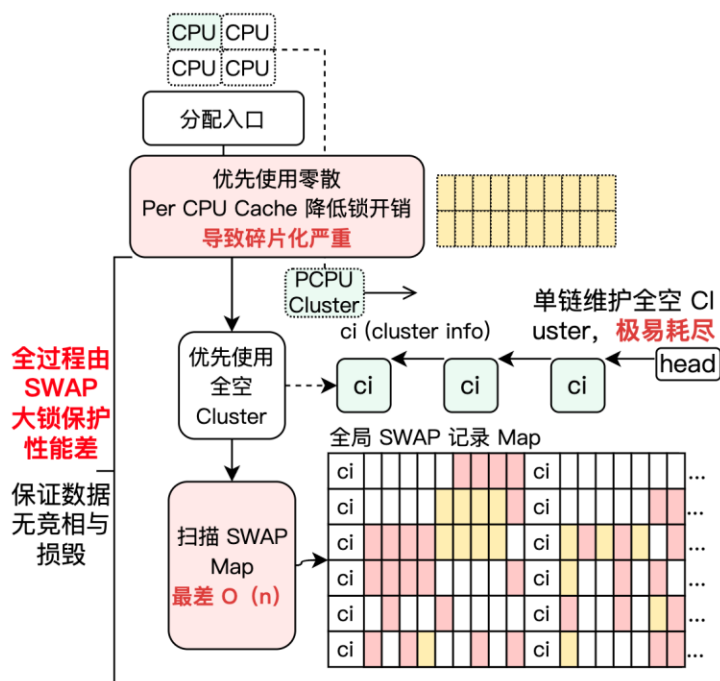


mTHP (64k) Swapout Fallback



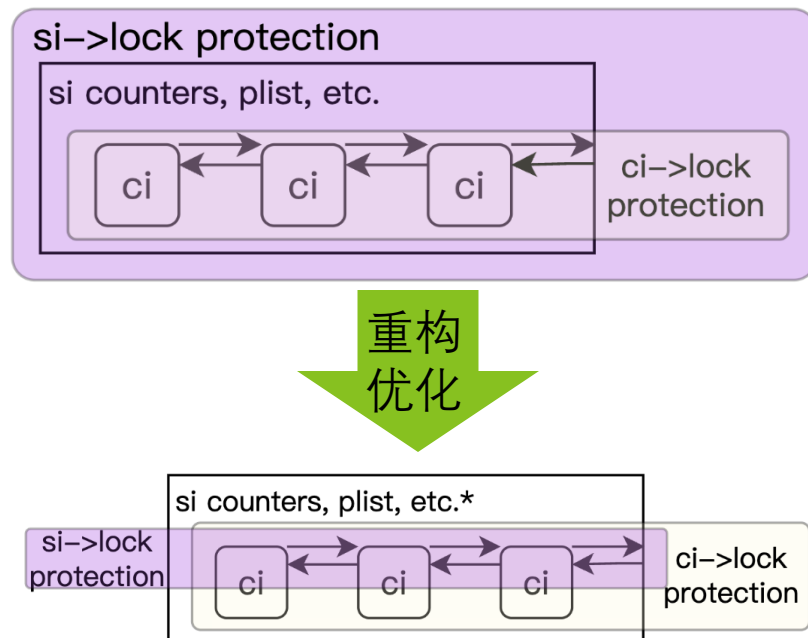
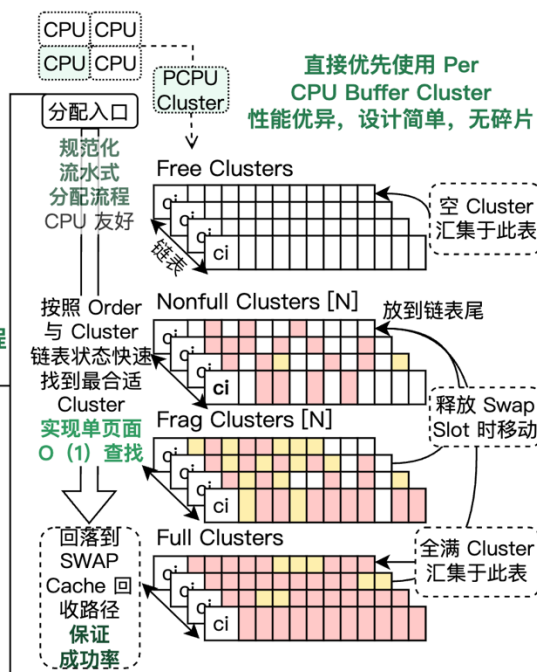
SWAP 锁细化

- SWAP 路径中最大的一个并发瓶颈在于设备（Swap Device 简称 si）锁，6.13 前 si 锁是全局大锁。
- 老分配器以 si 锁来保证大量 si 数据无竞相，只有部分热点可以使用 Cluster 级别锁（ci 锁）。
- 我们对整个 SWAP 分配路径进行重构，巧妙利用无锁算法以及内存屏障，去除了 si 锁瓶颈。
- 并进一步优化了 Per CPU SWAP Cluster 的使用方式，提供了更优异的并发设计。



重构优化

全过程无锁



老分配器 si 锁包裹 ci 锁，使得 si 锁难以避免（锁依赖）。新分配器重构整个锁流程，仅在一个 Cluster 耗尽时使用 si lock，其他场景全部使用小范围（2M）ci lock。

使用大量无锁算法，充分利用，并整理新分配器中 Cluster 管理流程，完成对 si 锁的解耦。包括全局计数器，SWAP 多设备优先级表，SWAP 设备卸载保护等全部无锁化。

SWAP 锁细化

- SWAP 路径中最大的一个并发瓶颈在于设备（Swap Device 简称 si）锁，6.13 前 si 锁是全局大锁。
- 老分配器以 si 锁来保证大量 si 数据无竞相，只有部分热点可以使用 Cluster 级别锁（ci 锁）。
- 我们对整个 SWAP 分配路径进行重构，巧妙利用无锁算法以及内存屏障，去除了 si 锁瓶颈。
- 并进一步优化了 Per CPU SWAP Cluster 的使用方式，提供了更优异的并发设计。

Using 4K folio by default, 64k mTHP and sequential access (!-R) results are also provided. 6 test runs for each case, Total Throughput:

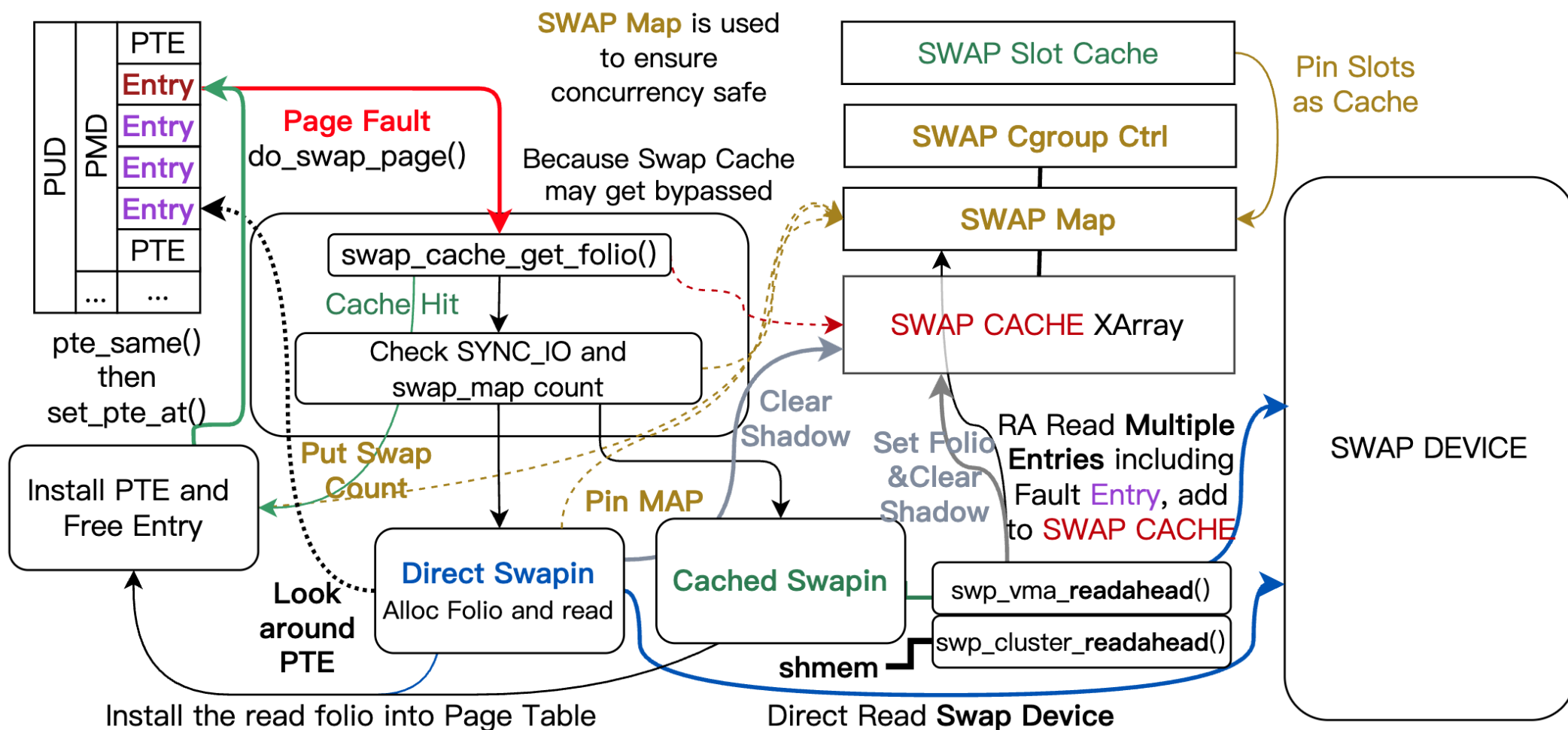
Test	Before (KB/s) (stdev)	After (KB/s) (stdev)	Delta
Random (4K):	69937.11 (16449.77)	369816.17 (24476.68)	+428.78%
Random (64k):	123442.83 (13207.51)	216379.00 (25024.83)	+75.28%
Sequential (4K):	6313909.83 (148856.12)	6419860.66 (183563.38)	+1.7%

- VM Scability SWAP 随机吞吐量提升 400%
- 我们的新 Swap Allocator 在 6.14 中完整合入主线替代了旧分配器。
- <https://lore.kernel.org/linux-mm/20241230174621.61185-1-ryncsn@gmail.com/T/>
- <https://lore.kernel.org/lkml/20250313165935.63303-1-ryncsn@gmail.com/T/>

make -j<NR> (NR / Mem / ZRAM)	System Time (seconds) (Before / After / Delta)		
With 4k pages only:			
6 / 192M / 3G	1533 /	1522 /	-0.7%
12 / 256M / 4G	2275 /	2226 /	-2.2%
24 / 384M / 5G	3596 /	3154 /	-12.3%
48 / 768M / 7G	8159 /	3605 /	-55.8%
96 / 1.5G / 10G	18541 /	6462 /	-65.1%
With 64k mTHP:			
24 / 512M / 5G	3585 /	3469 /	-3.2%
48 / 1G / 7G	8173 /	3607 /	-55.9%
96 / 2G / 10G	16305 /	7791 /	-52.2%

- 内核并发构建 SWAP 损耗降低达 60%
- sysbench 测试 MySQL 性能提升 40%
- Before: 6292.11 tps
- After: 8972.73 tps

- ## SWAP 流程



SWAP 流程

- 我们长期关注，并对 **SWAP** 和相关子系统在进行整体性完全审阅与重构。
- 期间的讨论中社区报告，我们修复了一个在上游隐藏 8 年之久的数据损毁问题：
 - 构建 Reproducer，被内核社区归为 CVE-2024-26759。
 - <https://lore.kernel.org/linux-mm/20240219082040.7495-1-ryncsn@gmail.com/>
- 由于 Swap Cache 的核心数据结构为 Xarray，Page Cache 的核心数据结构也是 Xarray，我们在重构期对 Page Cache / Xarray 算法进行了优化并修复疑难 Bug：
 - Bug 困扰了众多一线厂家数年；修复发往上游，解决了这个难以复现和分析的隐藏 Bug，同时性能提高 10%，获得 Jens Axboe，Linus Torvalds 等核心人员认可和业界关注。
 - <https://lwn.net/Articles/969950/>

```
* Known and unfixed active data loss bug in MM +
XFS with large folios since Dec 2021 (any kernel
from 6.1 upwards)
@ 2024-09-12 21:18 Christian Theune
  2024-09-12 21:55 ` Matthew Wilcox
    0 siblings, 1 reply; 81+ messages in thread
From: Christian Theune @ 2024-09-12 21:18 UTC
(permalink / raw)
To: linux-mm, linux-xfs@vger.kernel.org, linux-
fsdevel, linux-kernel
Cc: torvalds, axboe, Daniel Dao, Dave Chinner,
willy, clm,
    regressions, regressions

Hello everyone,

I'd like to raise awareness about a bug causing
data loss somewhere in MM interacting with XFS
that seems to have been around since Dec 2021
(https://github.com/torvalds/linux/commit/6795801366da0cd3d99e27c37f020a8f16714886)
```

社区在 2024 年 9 月时着重关注自 2021 年开始便时而浮现的缓存损毁导致系统宕机问题。

```
.. and that sounds like a good thing, except for the test coverage, I guess.

Which you seem to have fixed:

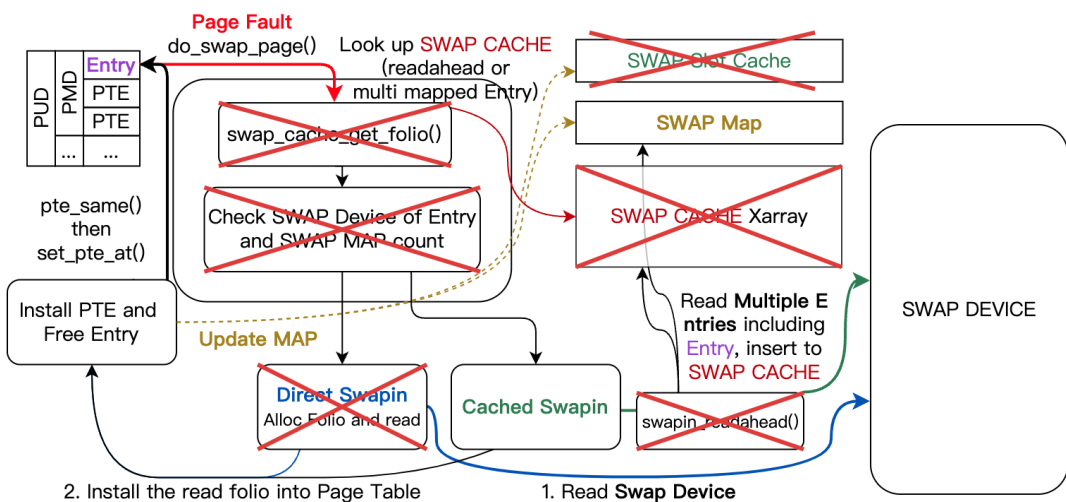
> With the changes in 6.10, we only get into that xas_destroy() case above
> when the conflicting entry is a shadow entry, so I changed my repro to
> use memory pressure instead of fadvise.
>
> I also added a schedule_timeout(1) after the split alloc, and with all
> of that I'm able to consistently make the xas_destroy() case trigger
> without causing any system instability. Kairui Song's patches do seem
> to have fixed things nicely.

<confused thumbs up / fingers crossed emoji>

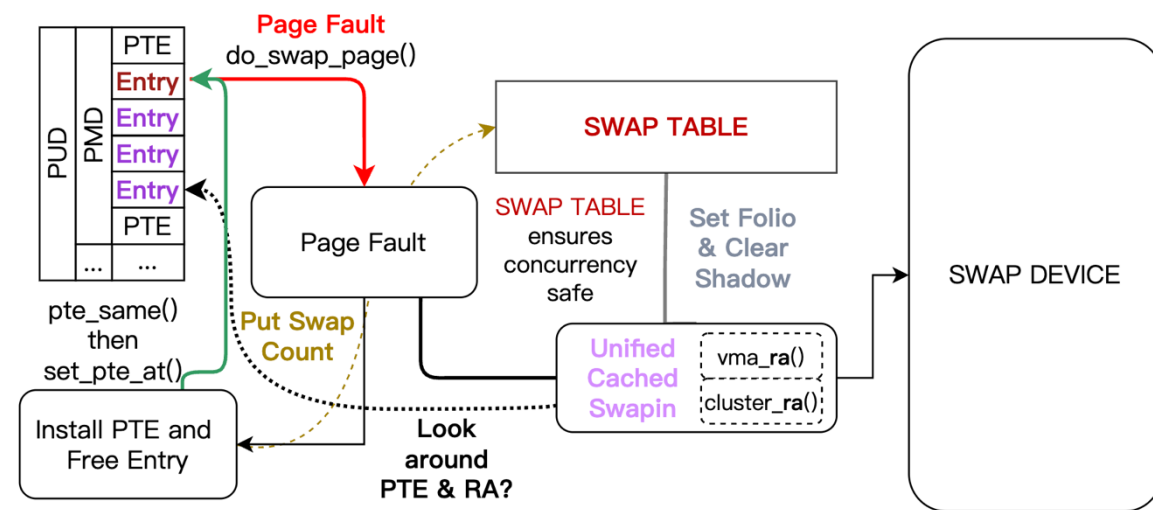
Linus
```

问题的引发与我们的修复引起了 Jens Axboe，Matthew Wilcox，Linus Torvalds 等的关注。

- 社区对整个 **SWAP** 的设计诟病已久，数年没有有效推进。
- 我们提出了新的 **SWAP Table** 概念，使用一个全新数据结构来表达与管理 **SWAP** 数据。
- 在 **LSFMM** 等会议上引起讨论和关注，这也会是 **SWAP** 下一个重点方向。
- 常见工作负载性能可以提升 10% 左右，同时内存占用，**SWAP** 内存压缩，**mTHP** 的通用分配与换入换出，**SSD** 写块适配等均可以此为基础展开。

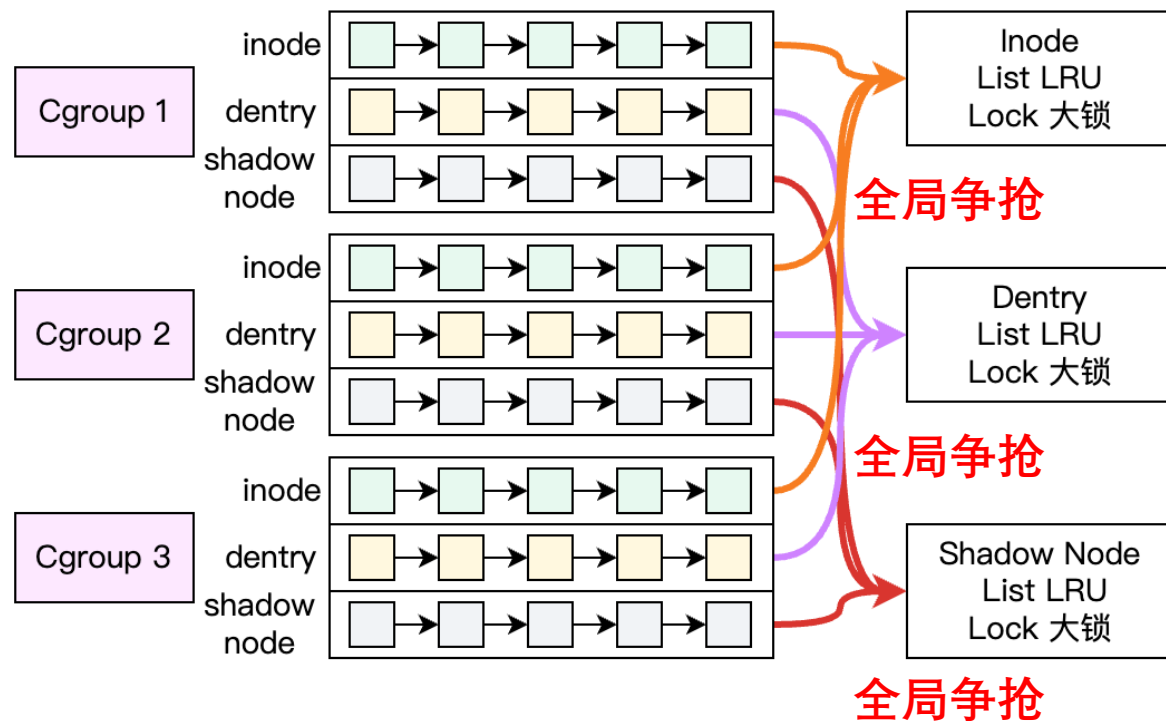


重构
优化



缓存淘汰效率

- 内核中除了页面管理与卸载淘汰机制外（LRU, SWAP），针对小于页面的对象也需要进行淘汰与管理（如 **Inode**, **Dentry**, **Shadow Node**）。
- 每个 Cgroup 都管理了自己的一套 List LRU 用于管理这些小对象，和在有内存压力时进行释放。
- 由于是分配路径负责将对象插入其对应的列表中（inode -> inode List LRU, dentry -> dentry List LRU）且涉及 Reparenting 等问题，同类型 List 一直由同一个全局锁进行保护。
- 在多个容器同时有内存压力进行回收时，会导致全局争抢。

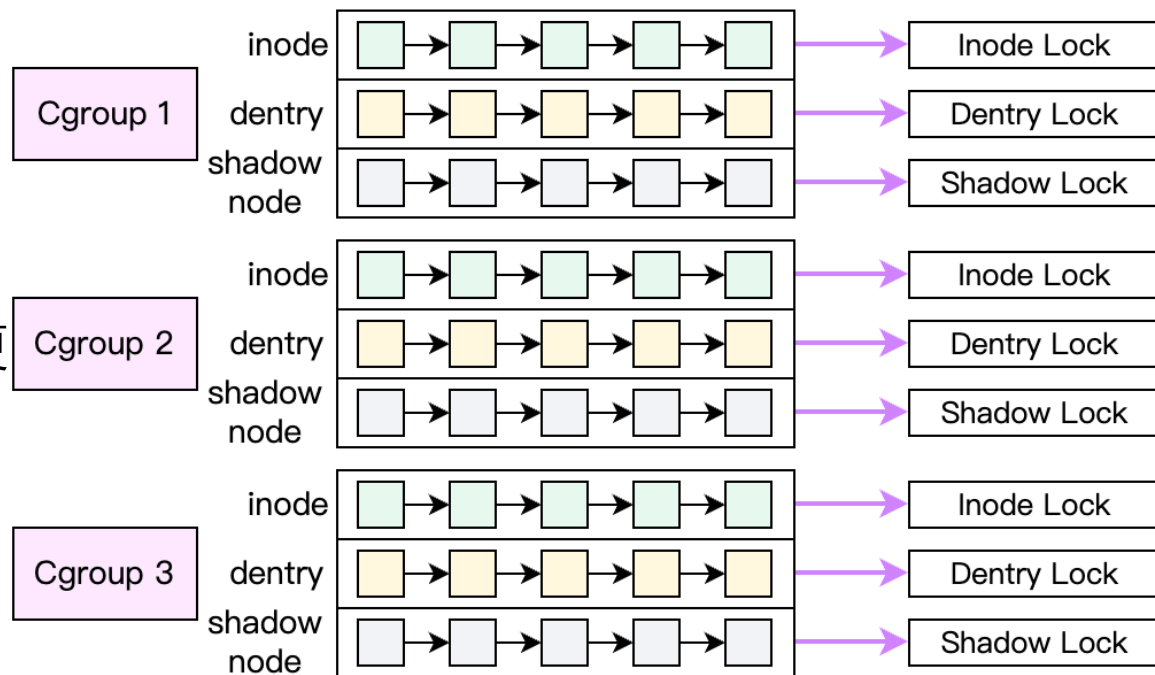


缓存淘汰效率

- 针对这一问题，我们也进行了专项优化。
- 充分利用 RCU 与锁的语义实现了分配与回收时 List LRU 的稳定化。
- 同时优化重构了 Reparent 过程，性能更好，代码更简洁。
- 将 List LRU 争抢拆解到了 Cgroup 级别。
- 以 Tar 压缩文件为例，内存压力场景下性能提升 15% 以上。

6.13 中已合入：

<https://lore.kernel.org/linux-mm/20241104175257.60853-1-ryncsn@gmail.com/>



基础性能与可扩展性 - 内存计数

- 内存页面计数与统计 (RSS) 是内核最基础核心能力，所有系统都离不开它，例如 top 命令。
- 即便是基本的计数器，也存在开销，是值得优化的部分。
- 内核曾经使用每个 Task Batch 多个计数事件的方式来避免频繁更新全局计数器，提高性能。
- 但这也极大降低了精度，且性能仍不理想，新版本内核中使用了 Per Task Per CPU 计数器。
- 这提高了性能，但又提高了内存开销和导致了大量并发 Task 启动时的性能问题。
- 我们于 2022 年提出了 Global Per CPU RSS Counter 设计，与调度系统结合。
- 实现了低内存消耗，极低的开销，高并发性能。

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
4701	root	20	0	3428	828	828	R	45.5	0.0	0:01.38	gzip
4699	root	20	0	8380	3156	3156	R	45.2	0.0	0:01.37	tar
4698	root	20	0	250116	246256	5576	S	17.8	0.2	0:00.54	python
4700	root	20	0	6812	796	796	S	4.3	0.0	0:00.13	tar
1	root	20	0	25460	13352	9428	S	2.3	0.0	0:05.38	systemd
1660	root	20	0	16192	7892	7124	S	1.3	0.0	0:00.17	systemd-logind
4349	root	20	0	7584	3192	3192	S	1.0	0.0	0:00.07	tmux: server
4585	root	20	0	9540	5316	3224	R	0.7	0.0	0:00.08	top
1653	dbus	20	0	5488	3124	2356	S	0.3	0.0	0:00.32	dbus-broker
1655	root	20	0	329140	17240	14936	S	0.3	0.0	0:00.63	NetworkManager
1659	root	20	0	16196	7164	6396	S	0.3	0.0	0:00.04	systemd-homed
3641	root	20	0	0	0	0	I	0.3	0.0	0:00.01	kworker/u386:1-kvfree_rcu_reclaim
4203	root	20	0	16104	7036	6268	S	0.3	0.0	0:00.01	systemd-userwor
4204	root	20	0	15940	6260	5492	S	0.3	0.0	0:00.01	systemd-userwor
4205	root	20	0	16104	7168	6400	S	0.3	0.0	0:00.01	systemd-userwor

经典的 top 命令，其中 RES，SHR，VIRT 均为内核内存计数器直接提供的信息。

```
pts/pgbench (100 Clients, 24 test run):
Before: 22556 TPS, 4.458ms latency
After: 23025 TPS, 4.347ms latency (~2% faster)

hackbench (32 process, 100 test run):
Before: took 48.774s
AfterA: took 48.540s (~1% faster)
```

```
- First, the accuracy of RSS of the demo C problem abo
USER      PID %CPU %MEM    VSZ   RSS TTY
root      1201 40.7  4.7  83245376 1498856 pts/0

Now it's exactly the amount of memory being consumed.

- Cache hit rate:
Booting up and login system (Fedora 36):
Hit/Miss (1 cache miss per 1800 events avg)::
1021027/567

Build linux kernel with tinyconfig and `make -j16`:
Hit/Miss (1 cache miss per 521 events avg):
35613357/68327
```

新设计性能更优，精确度高，内存开销低，仍在计划推进中。

<https://lwn.net/ml/linux-kernel/20220728204511.56348-1-ryncsn@gmail.com/>

展望未来



- 系统规模不断扩大，内存容量不断提升。
- Linux 内核中仍旧有众多需要提升的地方。
- 内核仍有潜力，有挑战，有机遇。
- 我们坚持上游合作，持续耕耘，回馈社区，合作共赢。

OpenCloudOS助力操作系统开源人才培养



OpenCloudOS 已有 **1053** 名学生开源爱好者加入社区，覆盖国内**81** 所学校，社区鼓励学生参与主线任务研发，培养出王鹏博、吴登智、王江奎、赵云等高校核心贡献者。

OpenCloudOS 高校开源人才培养全景图



▶ 与30+所高校落地合作，吸纳超过共同培育研究型人才、应用型人才、技能型人才

学生参与 OpenCloudOS 社区的方式



1. 下载 ISO 镜像，加入社区交流群

官网提供 OC8/OC9 镜像，推荐学生下载 DVD/Everything ISO 体验

2. 在官方 Gitee 仓库领取 ISSUE，提交开源社区贡献

实践任务由内核、测试、文档、AI、官网、Stream 等 SIG 小组导师发布，难度分低、中、高三个等级，导师把关代码质量，获得开源项目经验和经历

3. 组织交流活动推广 OpenCloudOS

在学校 Linux 俱乐部、社团、选修课上组织 OpenCloudOS 交流活动，社区提供导师、礼品支持。

4. 报名操作系统开源大赛等项目，参与 OpenCloudOS 主线任务研发

在 OpenCloudOS 社区导师的指导下，完成主线任务研发，成长为社区核心 Contributor。同时作为热心贡献者回答疑问和引导社区新人使用 OpenCloudOS。



北京航空航天大学



北京理工大学



华中科技大学



深圳技术大学

田震雄 西安电子科技大学

参与腾讯犀牛鸟开源人才培养计划 OpenCloudOS 社区的开源 issue 实战和开源课题实战项目。

并凭借卓越表现，获得「优秀学生」称号和《腾讯开源贡献者》荣誉证书。



采访稿：[OpenCloudOS 优秀贡献者-田震雄：勇敢迈出开源第一步](#)

华中科技大学 王江奎同学

参与开放原子开源大赛 OpenCloudOS 赛题，在社区专家陶松桥指导下，完成 OpenCloudOS8 至 OpenCloudOS9 迁移工具研发



采访稿：[OpenCloudOS 优秀贡献者-王江奎：参与开源是双向互动的过程](#)



贡献者证书



完成难度较高或社区重要任务，可获得社区周边奖励

高校参与 OpenCloudOS 社区案例



- 山东大学组织学生参与 OpenCloudOS 开源贡献

OpenCloudOS 联合山东大学网络与安全学院组织 OpenCloudOS 走进山东大学实践活动，吸引网络空间安全、电子信息、密码科学与技术等专业的**博士、硕士、本科637 名同学**参与。

组建 OpenCloudOS 山东大学开源俱乐部，包含 210 位同学。通过开源摘星 2023 OpenCloudOS WOW 项目，网络与安全学院的 23 名学生提交 151+ Pull request 被社区合并，涵盖文档 SIG、测试 SIG、Stream SIG 的 Issue 贡献。

- 北京大学开设 OpenCloudOS 选修课

OpenCloudOS 面向北京大学研究生一年级开设开源软件开发基础与实践选修课程。

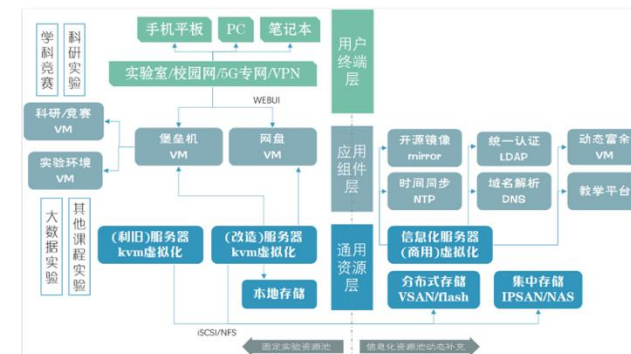
- 中南民族大学基于 OpenCloudOS 构建信息化数字基座

结合中南民族大学对教学实验环境的使用需求，融合了信息化基础设施资源池与基于 OpenCloudOS 构建的虚拟化资源池，打造了通用计算资源为载体的教学实验技术平台方案。目前正在基于 OpenCloudOS 建设教学实践课程。

- 江苏财经职业技术学院使用 OpenCloudOS 建设智慧校园

OpenCloudOS 针对高校的的实际的使用场景，为江苏财经职业技术学院等高校师生提供更好的支持和资源，提升校园网络、数据中心等方面的建设水平，满足师生的多元化需求。

同时，学校也积极参与到 OpenCloudOS 的开发和改进中，通过反馈和贡献，发挥示范作用，将 OpenCloudOS 的使用经验分享给其它高校，推动智慧校园建设的发展。



THANK YOU!

欢迎加入 OpenCloudOS 社区

共创 共享 共赢



申请成为 OpenCloudOS 社区伙伴



关注 OpenCloudOS 公众号